Data Science II Homework 1

Yongyan Liu (yl6107)

2026-02-15

Load and prepare the data first.

```
library(tidyverse)
library(caret)
library(glmnet)
library(pls)
library(corrplot)

# Load data
train_data <- read_csv("housing_training.csv") %>%
  janitor::clean_names()
test_data <- read_csv("housing_test.csv") %>%
  janitor::clean_names()

# Prepare data matrices for modeling
# Training data
x_train <- model.matrix(sale_price ~ ., train_data)[, -1]
y_train <- train_data$sale_price

# Test data
x_test <- model.matrix(sale_price ~ ., test_data)[, -1]
y_test <- test_data$sale_price
```

(a) Lasso Model

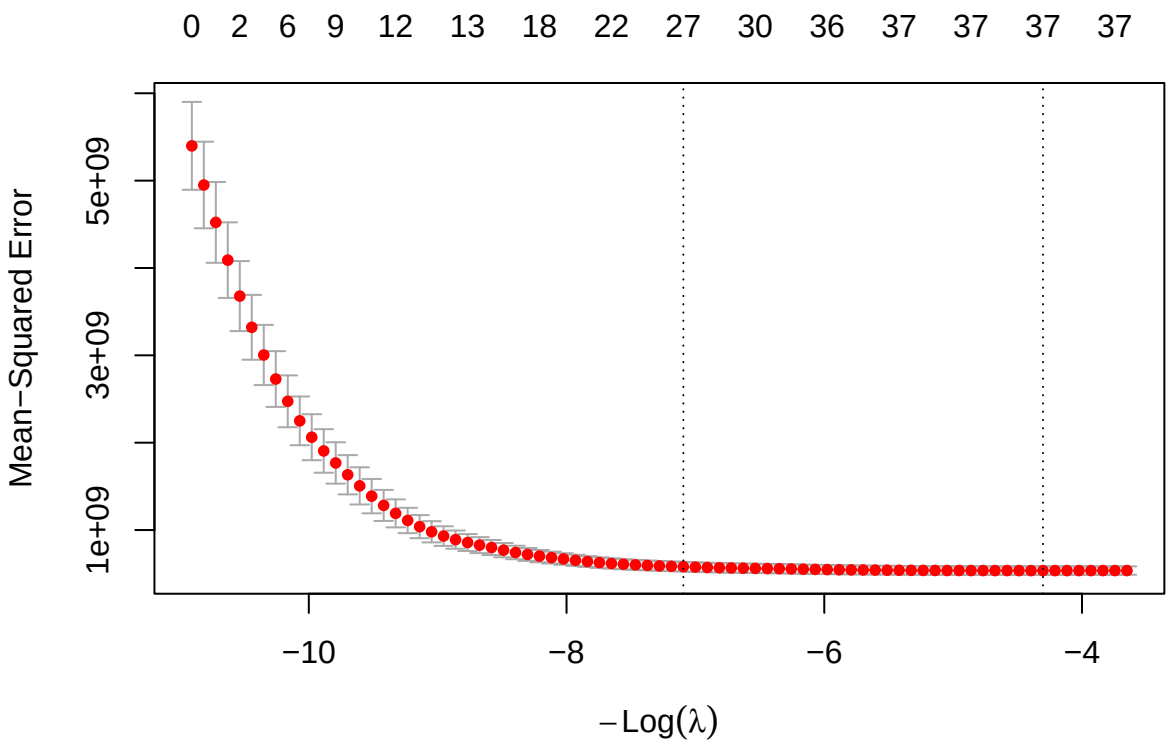
Question: Fit a lasso model on the training data. Report the selected tuning parameter and the test error. When the 1SE rule is applied, how many predictors are included in the model?

Solution

We fit the lasso model using the `glmnet` package with 10-fold cross-validation.

```
# Fit lasso model using cv.glmnet (alpha = 1 for lasso)
# Let glmnet choose lambda sequence automatically (data-driven, recommended)
set.seed(2026)
cv_lasso <- cv.glmnet(x_train, y_train,
                      alpha = 1,
                      nfolds = 10)
```

```r
# Plot CV curve with 1SE line
plot(cv_lasso)
```




```r
# Compute predictions and metrics for both lambda choices
lasso_pred_min <- predict(cv_lasso, newx = x_test, s = "lambda.min")
lasso_pred_1se <- predict(cv_lasso, newx = x_test, s = "lambda.1se")

lasso_mse <- mean((lasso_pred_min - y_test)^2)
lasso_mse_1se <- mean((lasso_pred_1se - y_test)^2)

coef_min <- coef(cv_lasso, s = "lambda.min")
coef_1se <- coef(cv_lasso, s = "lambda.1se")

n_predictors_min <- sum(coef_min[-1,1] != 0)
n_predictors_1se <- sum(coef_1se[-1,1] != 0)

# Summary table
lasso_results <- data.frame(
  Criterion = c("lambda.min", "lambda.1se"),
  Lambda = c(cv_lasso$lambda.min, cv_lasso$lambda.1se),
  Test_RMSE = c(sqrt(lasso_mse), sqrt(lasso_mse_1se)),
  Num_Predictors = c(n_predictors_min, n_predictors_1se)
)

knitr::kable(lasso_results, digits = 2,
```

```
            col.names = c("Criterion", "Lambda", "Test RMSE", "# Predictors"),
            caption = "Lasso Model: Comparison of Lambda Selection Criteria")
```

Table 1: Lasso Model: Comparison of Lambda Selection Criteria

Criterion	Lambda	Test RMSE	# Predictors
lambda.min	73.87	20955.73	37
lambda.1se	1203.93	20707.31	27

With 1SE rule: $\lambda = 1203.93$, resulting in **27 predictors**.

(b) Elastic Net Model

Question: Fit an elastic net model on the training data. Report the selected tuning parameters and the test error. Is it possible to apply the 1SE rule to select the tuning parameters for elastic net? If the 1SE rule is applicable, implement it to select the tuning parameters. If not, explain why.

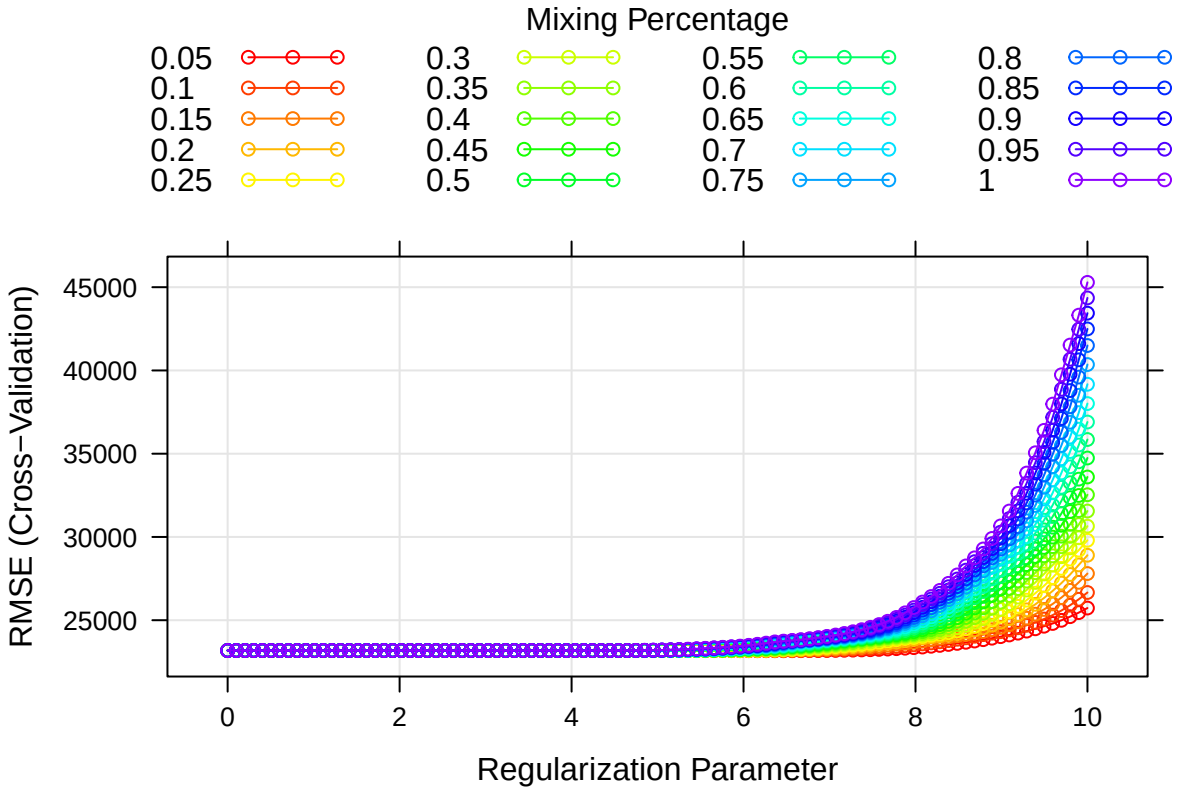
Solution

We use `caret` to tune both α and λ simultaneously via cross-validation.

```
# Set up cross-validation
ctrl <- trainControl(method = "cv", number = 10)

# Fit elastic net model
# Note: lambda_max depends on alpha (larger for smaller alpha), so we use a wide grid
set.seed(2026)
enet_fit <- train(x_train, y_train,
                  method = "glmnet",
                  tuneGrid = expand.grid(alpha = seq(0.05, 1, length = 20),
                                         lambda = exp(seq(0, 10, length = 100))),
                  trControl = ctrl)
```

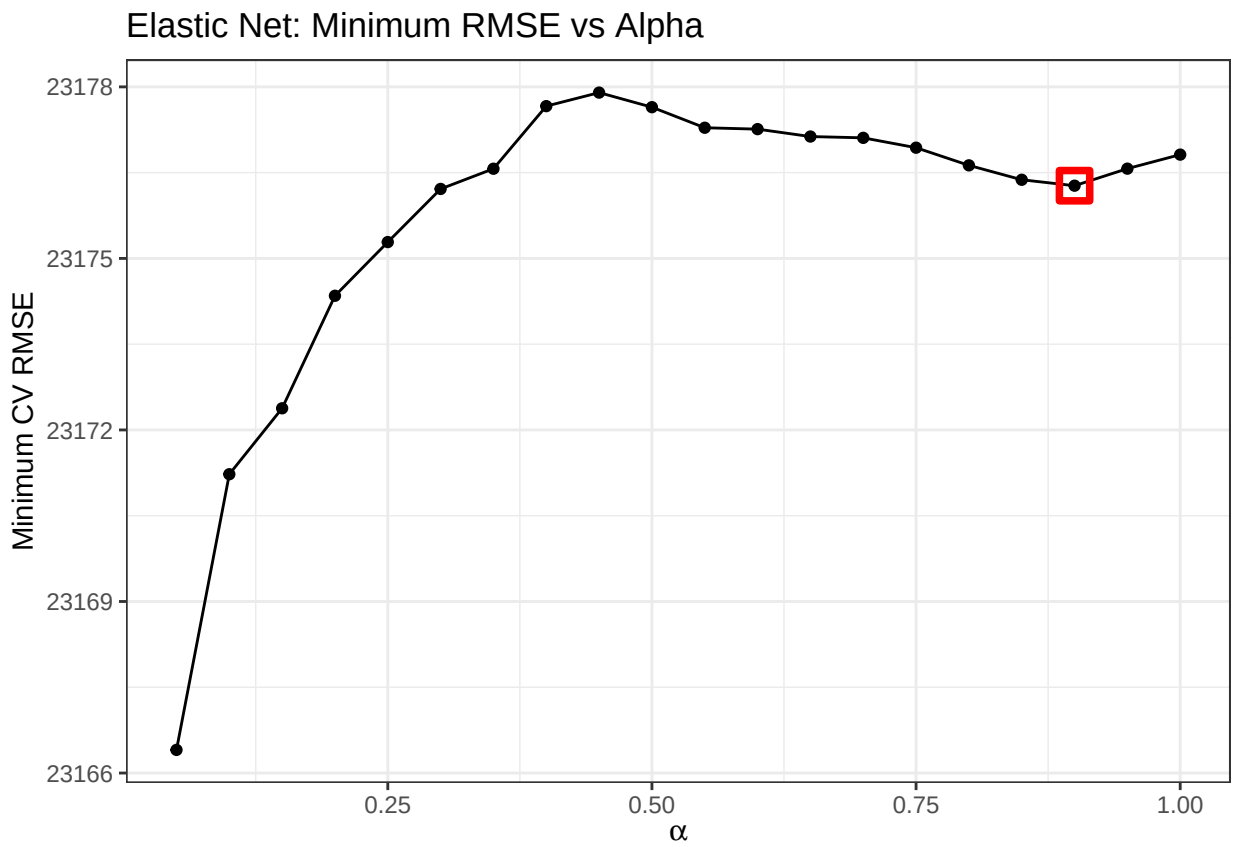
```
# Plot CV results
myCol <- rainbow(25)
myPar <- list(superpose.symbol = list(col = myCol),
              superpose.line = list(col = myCol))
plot(enet_fit, par.settings = myPar, xTrans = log)
```



The plot shows that smaller α values (ridge-like) have flatter CV curves. This is because ridge regression shrinks coefficients gradually without setting them to zero, making it less sensitive to λ. However, all α values achieve similar minimum RMSE at their optimal λ, indicating that the choice of α has minimal impact on prediction accuracy for this dataset.

```r
# Plot minimum RMSE for each alpha
min_rmse_by_alpha <- enet_fit$results %>%
  group_by(alpha) %>%
  summarize(min_RMSE = min(RMSE, na.rm = TRUE))

ggplot(min_rmse_by_alpha, aes(x = alpha, y = min_RMSE)) +
  geom_point() +
  geom_line() +
  geom_point(data = filter(min_rmse_by_alpha, alpha == 0.9),
             color = "red", size = 4, shape = 0, stroke = 2) +
  labs(x = expression(alpha), y = "Minimum CV RMSE",
       title = "Elastic Net: Minimum RMSE vs Alpha") +
  theme_bw()
```



The range of minimum RMSE across all α values is very small (11.5), indicating that the choice of α has minimal impact on prediction accuracy for this dataset. Since performance is similar, we prefer a **larger** α (closer to lasso) to obtain a **sparser, more interpretable model**. We select $\alpha = 0.9$ (marked with red box).

```
# Select alpha = 0.9 since all alphas perform similarly, prefer sparser model
best_alpha <- 0.9
best_lambda_enet <- enet_fit$results %>%
  filter(alpha == best_alpha) %>%
  slice_min(RMSE) %>%
  pull(lambda)

# Refit final model and compute test error
enet_final <- glmnet(x_train, y_train, alpha = best_alpha, lambda = best_lambda_enet)
enet_pred <- predict(enet_final, newx = x_test, s = best_lambda_enet)
enet_mse <- mean((enet_pred - y_test)^2)

coef_enet <- coef(enet_final, s = best_lambda_enet)
n_predictors_enet <- sum(coef_enet[-1,1] != 0)

# Results table
enet_results <- data.frame(
  Alpha = best_alpha,
  Lambda = best_lambda_enet,
  Test_RMSE = sqrt(enet_mse),
  Num_Predictors = n_predictors_enet
```

```
)
knitr::kable(enet_results, digits = 2,
             col.names = c("Alpha", "Lambda", "Test RMSE", "# Predictors"),
             caption = "Elastic Net: Selected Parameters and Test Error")
```

Table 2: Elastic Net: Selected Parameters and Test Error

Alpha	Lambda	Test RMSE	# Predictors
0.9	69.58	20985.83	37

1SE rule applicability: The 1SE rule is **not directly applicable** to elastic net because it has two tuning parameters (α and λ). The 1SE rule requires a single dimension to define "simpler" models (e.g., larger λ), but with a 2D grid there is no unique ordering of model complexity.

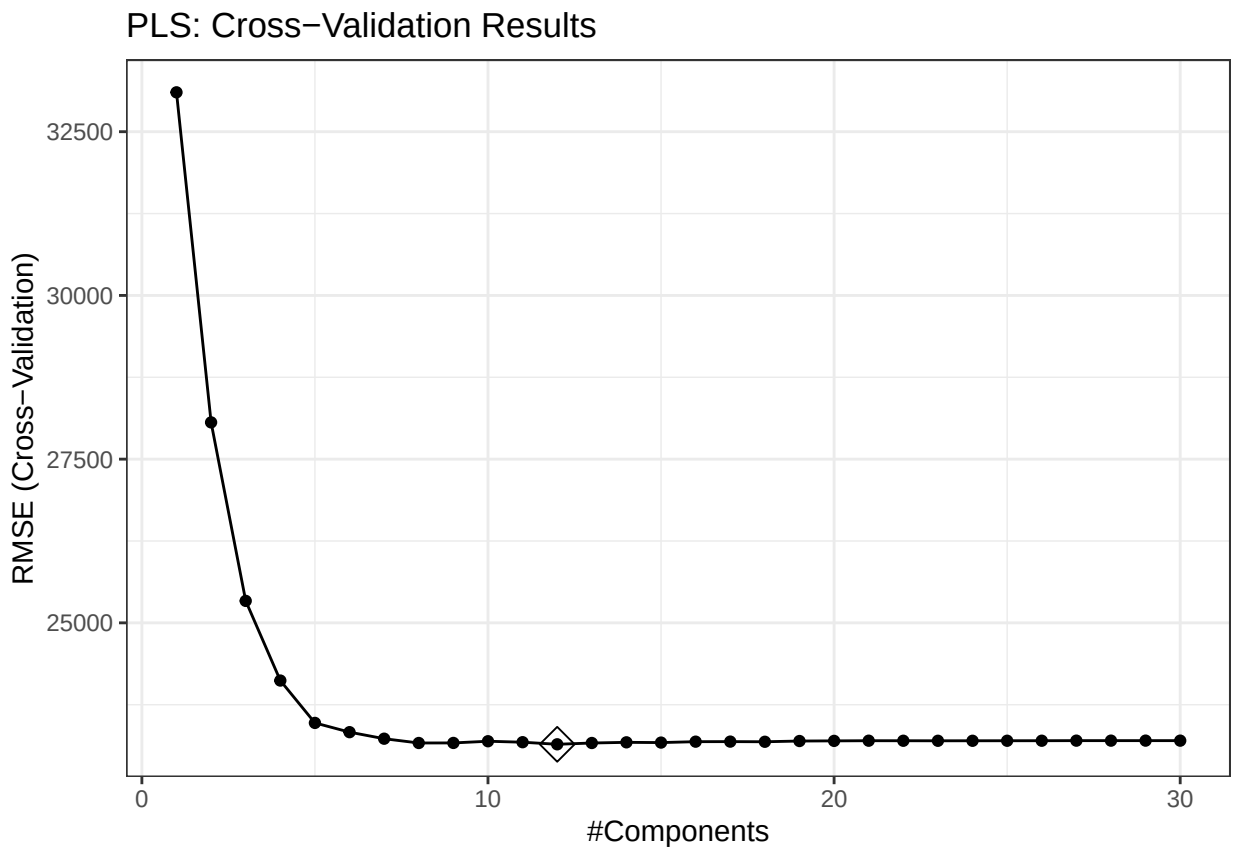
(c) Partial Least Squares Model

Question: Fit a partial least squares model on the training data and report the test error. How many components are included in your model?

Solution

```
# Fit PLS model (using ctrl from part b)
set.seed(2026)
pls_fit <- train(x_train, y_train,
                 method = "pls",
                 tuneGrid = data.frame(ncomp = 1:30),
                 trControl = ctrl,
                 preProcess = c("center", "scale"))

# Plot CV results
ggplot(pls_fit, highlight = TRUE) +
  theme_bw() +
  labs(title = "PLS: Cross-Validation Results")
```




```
# Best number of components
pls_fit$bestTune
```

```
##    ncomp
## 12    12
```

```
ncomp_best <- pls_fit$bestTune$ncomp

# Test error
pls_pred <- predict(pls_fit, newdata = x_test)
pls_mse <- mean((pls_pred - y_test)^2)
cat("Test RMSE:", round(sqrt(pls_mse), 2), "\n")
```

```
## Test RMSE: 21204.31
```

(d) Best Model Selection

Question: Choose the best model for predicting the response and explain your choice.

Solution

```r
# Summary of test errors for all models
test_results <- data.frame(
  Model = c("Lasso (min)", "Lasso (1se)", "Elastic Net", "PLS"),
  Test_RMSE = c(sqrt(lasso_mse), sqrt(lasso_mse_1se), sqrt(enet_mse), sqrt(pls_mse)),
  Num_Parameters = c(n_predictors_min, n_predictors_1se, n_predictors_enet, ncomp_best)
)
knitr::kable(test_results, digits = 2,
             col.names = c("Model", "Test RMSE", "# Parameters"),
             caption = "Test Error Comparison")
```

Table 3: Test Error Comparison

Model	Test RMSE	# Parameters
Lasso (min)	20955.73	37
Lasso (1se)	20707.31	27
Elastic Net	20985.83	37
PLS	21204.31	12

```r
best_model <- test_results$Model[which.min(test_results$Test_RMSE)]
```

The **Lasso (1se)** model is selected as the best model for predicting house sale price. It achieves the lowest test RMSE among all models. Also, we would prefer the one with fewer parameters for better interpretability and to reduce overfitting risk.

(e) glmnet vs. Caret Comparison for Lasso

Question: If R package "caret" was used for the lasso in (a), retrain this model using R package "glmnet", and vice versa. Compare the selected tuning parameters between the two software approaches. Should there be discrepancies in the chosen parameters, discuss potential reasons for these differences.

Solution

In part (a), we used `glmnet`. Now we fit lasso using `caret` and compare results.

```r
# caret approach (using same lambda sequence as glmnet for fair comparison)
set.seed(2026)
lasso_caret <- train(x_train, y_train,
                     method = "glmnet",
                     tuneGrid = expand.grid(alpha = 1, lambda = cv_lasso$lambda),
                     trControl = ctrl)

# Get caret's best lambda and compute 1SE equivalent
caret_results <- lasso_caret$results
caret_best_idx <- which.min(caret_results$RMSE)
caret_best_rmse <- caret_results$RMSE[caret_best_idx]
# cv.glmnet uses SE = SD/sqrt(nfolds) for 1SE rule
caret_best_se <- caret_results$RMSESD[caret_best_idx] / sqrt(10)
```

```r
lambda_caret_min <- caret_results$lambda[caret_best_idx]
# 1SE rule: largest lambda (simplest model) with RMSE within 1 SE of minimum
within_1se <- caret_results$RMSE <= caret_best_rmse + caret_best_se
lambda_caret_1se <- max(caret_results$lambda[within_1se])

# Predictions and coefficients for caret
pred_caret_min <- predict(lasso_caret, newdata = x_test)
coef_caret_min <- coef(lasso_caret$finalModel, lambda_caret_min)
coef_caret_1se <- coef(lasso_caret$finalModel, lambda_caret_1se)

mse_caret_min <- mean((pred_caret_min - y_test)^2)
pred_caret_1se <- predict(lasso_caret$finalModel, newx = x_test, s = lambda_caret_1se)
mse_caret_1se <- mean((pred_caret_1se - y_test)^2)

# Comparison table
comparison_results <- data.frame(
  Method = c("glmnet (min)", "glmnet (1se)", "caret (min)", "caret (1se)"),
  Lambda = c(cv_lasso$lambda.min, cv_lasso$lambda.1se, lambda_caret_min, lambda_caret_1se),
  Test_RMSE = c(sqrt(lasso_mse), sqrt(lasso_mse_1se), sqrt(mse_caret_min), sqrt(mse_caret_1se)),
  Num_Predictors = c(n_predictors_min, n_predictors_1se,
                     sum(coef_caret_min[-1,1] != 0), sum(coef_caret_1se[-1,1] != 0))
)
knitr::kable(comparison_results, digits = 2,
             col.names = c("Method", "Lambda", "Test RMSE", "# Predictors"),
             caption = "Lasso: glmnet vs caret Comparison")
```

Table 4: Lasso: glmnet vs caret Comparison

Method	Lambda	Test RMSE	# Predictors
glmnet (min)	73.87	20955.73	37
glmnet (1se)	1203.93	20707.31	27
caret (min)	61.33	20984.86	37
caret (1se)	910.73	20544.17	29

Discussion of Discrepancies:

The table shows that `glmnet` and `caret` select different lambda values despite using the same lambda grid. The discrepancies arise from:

1. **Different CV fold assignments**: `caret` and `cv.glmnet` use different internal randomization for creating CV folds, even with the same seed.
2. **Different RMSE aggregation**: `cv.glmnet` computes $\sqrt{\text{mean}(MSE)}$ while `caret` computes $\text{mean}(RMSE)$. These differ due to the order of operations.

Despite these differences, both methods yield similar test RMSE and number of predictors, indicating that the practical impact is minimal for this dataset.